

Glossaire : tous les termes techniques du PowerPoint ColorRoom

Fiche de récap pour l'oral E6 · Téo Tromprier (E2). Chaque terme = **définition simple** + ► **un exemple réel du projet (une ligne de code)**. Tout est tiré du dépôt [NewGenesisAgency/color_room](#) (branche `main`).

★ Sections à relire en priorité : **5** (variables), **6** (sécurité), **7 + 13** (IA Puissance 4).

1. Mots de vocabulaire (les « mots savants »)

- **Paradigme** : une *façon de penser/organiser* le code. Node-RED = paradigme **flux** ; React = paradigme **composants**. ► `export default function GamePuissance4() { ... }` *(un composant, pas un flux)*
- **Flow-based / dataflow** : « basé sur le flux » : on relie des **nœuds** par des fils. C'est Node-RED (écarté). ► à l'inverse, j'appelle une fonction : `makeMove(col)`
- **Déclaratif** : je **décris le résultat** voulu, pas les étapes. ► `<div className="cell" onClick={() => makeMove(c)} />`
- **Diffable** : fichier dont on **voit les différences** ligne à ligne dans Git. ► `git diff app/lib/auth.ts`
- **Versionnable** : suivi **version par version** (historique, retour arrière). ► `git commit -m "feat(ia): minimax alpha-beta"`
- **Idempotent** : rejouable **sans rien casser ni dupliquer**. ► `CREATE TABLE IF NOT EXISTS crg_users (...)`
- **Heuristique** : **règle approximative** qui note vite une situation. ► `if (me === 3) return 130;`
- **Atomique** : **insécable** (tout ou rien). ► `const insertAll = db.transaction(() => { ... })`
- **Concurrence** : plusieurs tâches **en même temps** à coordonner. ► `if (hwInFlight < HW_CONCURRENCY) hwInFlight++;`
- **Volatile** : **en mémoire vive**, perdu au rechargement (voir §5). ► `const comboRef = useRef(0)`
- **Persistant** : **conservé sur disque**, survit au redémarrage. ► `db.prepare('INSERT INTO crg_scores ...').run(...)`

2. Front-end · React / Next.js

- **Composant** : **brique d'UI réutilisable**. ► `export default function GamePuissance4(props: GameTileProps) { ... }`
- **JSX / TSX** : syntaxe **HTML + code** dans un composant ; TSX = version **TypeScript** (`.tsx`). ► `return <div className="board">{cells}</div>;`
- **DOM** : l'**arbre des éléments** affichés par le navigateur. ► `container.appendChild(renderer.domElement)`
- **Virtual DOM** : copie mémoire ; React ne met à jour **que ce qui change**. ► `setScore(s => s + pts)` * (déclenche le diff)*
- **Réconciliation** : le **calcul de la différence** entre deux Virtual DOM. ► `setGrid(next)` *(React applique le minimum de changements)*
- **État réactif** : données qui **redessinent l'UI** quand elles changent. ► `const [score, setScore] = useState(0)`
- **useState** : variable **réactive**. ► `const [phase, setPhase] = useState<Phase>('ready')`

- **useRef** : variable mémoire **sans re-rendu** (volatile, rapide). ▶ `const lightRef = useRef(0)`
- **Next.js** : framework qui réunit **front + back** en un projet. ▶ `import { NextRequest, NextResponse } from 'next/server'`
- **App Router** : chaque **dossier = une URL**. ▶ fichier `app/app/api/auth/login/route.ts` → URL `/api/auth/login`
- **Route Handler** : une **route d'API**. ▶ `export async function POST(req: NextRequest) { ... }`
- **Server Component** : composant **rendu côté serveur** (par défaut dans l'App Router). ▶ `export default async function Page() { ... }`
- **SSR** : **rendu serveur** avant envoi au navigateur. ▶ (Next.js, automatique pour les Server Components)
- **Three.js / WebGL** : **3D dans le navigateur** (carte graphique). ▶ `new THREE.WebGLRenderer({ antialias: true, alpha: true })`

3. TypeScript · qualité du code

- **TypeScript** : JavaScript **avec des types**. ▶ `function scoreWindow(me: number, opp: number): number`
- **Typage statique** : types **vérifiés à la compilation**. ▶ `const classRow = stmt.get(code) as { id: string } | undefined`
- **Compilation / transpilation** : traduire TS → JS (et détecter les erreurs). ▶ `npx tsc --noEmit`
- **tsc** : le **compilateur TypeScript**. ▶ `tsc --noEmit` *(0 erreur = build sûr)*
- **ESLint** : **analyseur** de mauvaises pratiques. ▶ `npm run lint`
- **Runtime** : l'**environnement d'exécution** (Node.js). ▶ `node ./node_modules/next/dist/bin/next start`

4. Base de données · SQLite

- **SQLite** : base **embarquée** = un simple **fichier**. ▶ `COLOR_ROOM_DB_PATH=/data/ColorRoomDB.db`
 - **better-sqlite3** : pilote SQLite **synchrone**. ▶ `db.prepare('SELECT * FROM crg_games').all()`
 - **Requête préparée** : paramètres **?** séparés des valeurs (**anti-injection**). ▶ `db.prepare('SELECT id FROM crg_users WHERE name = ?').get(name)`
 - **Injection SQL** : attaque par SQL malveillant ; **bloquée** par le **?**. ▶ `... WHERE name = ?` *(jamais de concaténation)*
 - **WAL** : **lire pendant qu'on écrit** (concurrency). ▶ `db.pragma('journal_mode = WAL')`
 - **busy_timeout** : SQLite **réessaie** au lieu d'échouer. ▶ `db.pragma('busy_timeout = 5000')`
 - **Clé étrangère** : lien entre tables + suppression en cascade. ▶ `FOREIGN KEY(user_id) REFERENCES crg_users(id) ON DELETE CASCADE`
 - **Migration** : création/MAJ du schéma au démarrage (**idempotente**). ▶ `CREATE TABLE IF NOT EXISTS crg_scores (...)`
 - **Singleton** : **une seule** connexion partagée. ▶ `db.pragma('foreign_keys = ON')` *(exécuté une fois, dans `getDb()`)*
 - **ERD** : schéma des **tables et relations**. ▶ tables `crg_users`, `crg_classes`, `crg_scores`, `crg_sessions` ...
-

5. Variables & persistance (les questions « pointues »)

- **Transaction** : groupe d'opérations en **un seul bloc**. ▶ `db.transaction(() => { ... })`
- **ACID** : Atomicité, Cohérence, Isolation, Durabilité. ▶ `insertAll() // BEGIN ... COMMIT`
- **Variable transactionnelle** : `BEGIN ... COMMIT` (+ **ROLLBACK** si erreur). ▶ `const insertAll = db.transaction(() => { db.prepare("INSERT INTO crg_users ...").run(...); ... })`
- **ROLLBACK** : **annulation totale** si une étape échoue. ▶ une exception dans `db.transaction(...)` annule tout automatiquement
- **Variable atomique** : valeur partagée modifiée de façon **insécable** (atomique car Node mono-thread). ▶
`let hwInFlight = 0;`
- **Sémaphore** : **compteur** qui borne les accès simultanés (2 max). ▶ `const HW_CONCURRENCY = 2;`
- **Mono-thread** : Node exécute sur **un seul fil** → pas de conflit sur `hwInFlight`. ▶ `await new Promise(r => setTimeout(r, 5))`
- **Boucle d'événements** : enchaîne les tâches une par une. ▶ `async function acquireHwSlot(): Promise<boolean> { ... }`
- **Promise** : une **valeur future** (asynchrone). ▶ `return new Promise<boolean>((resolve) => hwWaiters.push({ resolve })))`
- **Promise.all** : plusieurs tâches **en parallèle**. ▶ `const results = await Promise.all(allRequests.map(r => sendOne(r)))`
- **File d'attente (queue)** : les requêtes en trop **patientent**. ▶ `const hwWaiters: Waiter[] = [];`
- **Variable d'environnement** : réglage **hors code**, hors Git. ▶ `process.env.SUPERVISION_API_URL`

Zoom : les variables volatiles (à bien savoir)

Une variable **volatile** vit **en mémoire vive** (RAM) le temps d'une session : elle est **rapide** mais **perdue** dès qu'on recharge la page ou qu'on éteint. C'est l'opposé d'une variable **persistante** (écrite en base SQLite).

Dans mon projet, j'utilise du volatile pour l'**état runtime des jeux** (le combo en cours, la dalle allumée, le chrono) : ça change des dizaines de fois par seconde, **inutile de l'écrire sur disque**. Seul le **résultat final** (le score) devient persistant.

- **Volatile (mémoire, pas de re-rendu)** ▶ `const comboRef = useRef(0)` puis `comboRef.current++` * (GameColorSpeed.tsx)*
- **Volatile réactif (mémoire + re-rendu de l'UI)** ▶ `const [score, setScore] = useState(0)`
- **Persistant (survit à tout, sur disque)** ▶ `db.prepare('INSERT INTO crg_scores (user_id, game_id, score) VALUES (?, ?, ?)').run(uid, gid, score)`

Phrase pour le jury : « L'état d'un jeu est **volatile** (`useRef` / `useState` , en RAM) : rapide et jetable. Je ne **persiste** en SQLite que ce qui doit survivre, comme le score. »

6. Sécurité · RGPD

- **Hachage** : empreinte **irréversible** d'un mot de passe. ▶ `const hash = pbkdf2Sync(password, salt, 100_000, 64, 'sha512')`
- **PBKDF2** : hachage **volontairement lent** (répétitions). ▶ `pbkdf2Sync(password, salt, 100_000, 64, 'sha512')`
- **HMAC / SHA-512** : la fonction de base utilisée. ▶ `... , 'sha512')`
- **Itérations** : nombre de répétitions (coût anti-bruteforce). ▶ `100_000`
- **Sel (salt)** : aléatoire **unique** par compte. ▶ `const salt = randomBytes(16).toString('hex')`

- **Rainbow table** : table d'empreintes pré-calculées ; le **sel** la rend inutile. ► `return \ ${salt}:${hash}}\` *(sel stocké avec le hash)*`
- **Cookie** : porte le **jeton de session**. ► `res.cookies.set('crg_session', token, { httpOnly: true, sameSite: 'lax' })`
- **HttpOnly** : cookie **inaccessible au JS** (anti-XSS). ► `{ httpOnly: true }`
- **SameSite=lax** : pas envoyé depuis un autre site (anti-CSRF). ► `{ sameSite: 'lax' }`
- **XSS** : injection de JS malveillant ; **mitigée** par HttpOnly. ► `httpOnly: true`
- **CSRF** : action à l'insu de l'utilisateur ; **mitigée** par SameSite. ► `sameSite: 'lax'`
- **Session / jeton** : identifiant aléatoire (30 j). ► `const token = randomBytes(32).toString('hex')`
- **RGPD** : minimisation + effacement en cascade. ► `FOREIGN KEY(user_id) REFERENCES crg_users(id) ON DELETE CASCADE`

7. IA du Puissance 4 (résumé · détails en §13)

- **Minimax** : explore l'**arbre des coups**, MAX pour l'IA / MIN pour l'adversaire. ► `minimax(d.grid, depth-1, alpha, beta, false)`
- **Élagage alpha-bêta** : **coupe** les branches inutiles. ► `if (alpha >= beta) break; // coupure bêta`
- **Profondeur** : nombre de coups anticipés (1 → 12). ► `legendaire: { depth: 12, ... }`
- **Fenêtre de 4** : chaque alignement possible de 4 cases. ► `score += scoreWindow(me, opp)`
- **Centralité** : bonus aux colonnes du centre. ► `const COL_WEIGHT = [1, 2, 4, 7, 4, 2]`
- **Anti-piège** : la **défense prime** l'attaque. ► `if (opp === 3) return -170; *(vs if (me === 3) return 130; )*`

8. Réseau · multijoueur

- **Hors-ligne (offline)** : tout local (Pi + SQLite + matériel). ► `env_file: [{ path: './app/.env', required: false }]`
- **Wi-Fi local** : le Pi diffuse **ColorRoom_WiFi**. ► accès `http://172.17.40.39/`
- **Proxy** : le navigateur passe par une **route relais**. ► `POST /api/supervision/batch`
- **WebSocket** : temps réel bidirectionnel (**non utilisé ici**). ► remplacé par du polling (voir ci-dessous)
- **Polling** : on **interroge** `/state` régulièrement. ► `setInterval(() => fetch('/api/multiplayer/state'), 700)`
- **Heartbeat** : signal de **présence**. ► `updated_at` rafraîchi à chaque appel d'état
- **UUID / id aléatoire** : identifiant unique. ► `randomBytes(16).toString('hex')`
- `state_json` : état de la partie **sérialisé en base**. ► `SELECT ... state_json FROM crg_mp_sessions WHERE status = 'active'`
- **Code de salon (multijoueur)** : code court pour **rejoindre une partie** (≠ code de classe). ► `room_code` dans `crg_mp_sessions`
- **Code de classe + QR code** : code à 6 caractères (ex. `CS5VHX`, sans 0/O 1/I) pour **rejoindre une classe** ; doublé d'un **QR code**. ► saisi à l'inscription : `body.classCode`
- **CORS** (*Cross-Origin Resource Sharing*) : règle qui **bloque** par défaut les appels d'un site vers une autre origine ; il a fallu **autoriser** l'API Supervision. ► en-têtes `Access-Control-Allow-Origin`
- **portproxy / pare-feu (Windows)** : exposer l'API locale (8080) sur le réseau (18080). ► `netsh interface portproxy add v4tov4 listenport=18080 connectport=8080`

- **AbortController** : **annule** une requête trop longue (timeout). ► `forceAbortCtrl.abort()`;
`forceAbortCtrl = new AbortController()`

9. Colorimétrie · matériel (la ColorRoom)

- **Cellule (salle)** : la ColorRoom compte **2 cellules jumelles** ; chaque cellule est tapissée de plaques sur les murs ET le **plafond**. ► **42 plaques** au total (21/cellule)
- **RS-485** : **bus série** robuste qui pilote les 42 plaques (côté matériel). ► piloté par `supervision.exe` , appelé via `POST /api/supervision/batch`
- **Canal (LED)** : une commande de couleur ; **32 par plaque**. ► `for (let i = 1; i <= 32; i++) { ... }`
- **Spectre / longueur d'onde (nm)** : la « recette » de la lumière. ► 24 canaux à spectre étroit (≈404 → 780 nm) + 8 blancs
- **Colorimètre CS-150/160** : mesure la lumière (Konica Minolta). ► lu via `GET /api/cs160`
- **Tristimulus XYZ (CIE)** : 3 valeurs décrivant la couleur perçue. ► réponse `{ X, Y, Z, Lv, x, y }` de `/api/cs160`
- **CIE 1931** : le diagramme standard (« fer à cheval »). ► composant `ChromaticityDiagram.tsx`
- **Chromaticité (x, y)** : la couleur **sans la luminosité**. ► `{ x, y }` placés sur le diagramme
- **Luminance Lv** : intensité perçue en **cd/m²**. ► champ `Lv` de la mesure
- **ΔE (Delta E)** : **écart** entre deux couleurs (score de précision). ► `const deltaE = Math.hypot(dx, dy, dz) *(principe)*`
- **Remappage des canaux** : couleur **RGB** → **32 canaux**. ► envoi via `/api/supervision/batch` (un objet par dalle)
- **Pont .NET** : relie mon API au colorimètre. ► `CS160_API_URL` (lu dans `process.env`)

10. Infrastructure · DevOps

- **Docker** : emballe l'app dans une **image** lancée en **conteneur**. ► `docker compose up -d --build`
 - **Conteneur** : instance qui tourne. ► `docker compose ps`
 - **Image multi-stage** : Dockerfile en étapes (image finale légère). ► `FROM node:20-slim AS builder`
 - **arm64** : architecture du **Raspberry Pi**. ► image construite pour `linux/arm64`
 - **Volume** : dossier **persistant** monté dans le conteneur. ► `volumes: [". /app/data:/data"]`
 - **Portainer** : supervision web des conteneurs. ► `image: portainer/portainer-ce:2.21.4`
 - **CI/CD** : test → build → deploy **automatiques**. ► `.gitlab-ci.yml` : `stages: [pretest, test, build, deploy]`
 - **Git** : gestion de versions (commits, branches). ► `git push -u origin ux-last`
 - **Branche / merge** : je développe sur `ux-last` (intégration) puis je **fusionne sur main** (stable). ► `git checkout main && git merge ux-last`
 - **Méthode agile** : développement **incrémental**, livraisons régulières, **échanges réguliers avec le client** (M. Labayrade, directeur BPMNP). ► itérations + retours client
 - **Kanban** : tableau de suivi des tâches en colonnes **À faire / En cours / Terminé** ► visualise l'avancement de l'équipe
 - **CSV · UTF-8 · BOM** : export tableur lisible par Excel. ► `const csv = '' + rows.join('\n')` *(le BOM force les accents)*
 - **Ollama / Gemini** : IA **locale** (hors-ligne) / IA cloud (optionnelle). ► `OLLAMA_URL=http://ollama:11434`
-

11. UML (les diagrammes) · artefact à montrer

- **UML** : langage de **schémas** standard. ▶ 15 diagrammes dans `scripts/uml/`
 - **Cas d'utilisation** : acteurs + fonctions. ▶ `ColorRoom_UseCases.png`
 - **Diagramme de classes** : conception **objet**. ▶ `ColorRoom_Classes.png`
 - **Diagramme de composants** : grands blocs logiciels. ▶ `ColorRoom_Composants.png`
 - **Diagramme de déploiement** : matériel réel. ▶ `ColorRoom_Deploiement.png`
 - **Diagramme de séquence** : dialogue chronologique. ▶ `ColorRoom_Seq_Auth.png`
 - **Diagramme d'états** : états + transitions. ▶ `ColorRoom_Etats_Jeu.png`
 - **Diagramme d'activité** : enchaînement d'actions. ▶ `ColorRoom_Activite_Remap.png`
 - **Gantt** : planning du projet. ▶ slide 8 du deck
-

12. Les 4 extraits de code montrés dans le deck

A. Variable transactionnelle (ACID) ▶ `.../main/app/app/api/auth/register/route.ts#L47-L62`

`db.transaction(() => { ... })` crée l'utilisateur **et** son adhésion de classe : **tout-ou-rien** (ROLLBACK si erreur).

À dire : « Une **variable transactionnelle** : `BEGIN ... COMMIT`, ROLLBACK automatique. »

B. Variable atomique · sémaphore ▶ `.../main/app/app/api/supervision/batch/route.ts#L39-L80`

`hwInFlight` borne les accès matériel à 2 ; les autres patientent dans une file de Promises. **Atomique** car Node est mono-thread.

À dire : « Je **borne la concurrence** avec un compteur atomique. »

C. Hachage PBKDF2 ▶ `.../main/app/lib/auth.ts#L19-L23`

`pbkdf2Sync(password, salt, 100_000, 64, 'sha512')`, stocké `sel:hash` ; jamais de mot de passe en clair.

À dire : « PBKDF2-HMAC-SHA512, lent et salé : anti rainbow-table. »

D. Évaluation minimax ▶ `.../main/app/app/_components/GamePuissance4.tsx#L118-L129`

`scoreWindow(me, opp)` note chaque fenêtre de 4 ; **défense (-170) > attaque (+130)** ; nourrit le minimax alpha-bêta.

À dire : « Heuristique par fenêtres de 4, défense pondérée plus fort, dans un minimax alpha-bêta. »

13. Focus : l'IA du Puissance 4 & les réseaux de neurones

⚠ Le point le plus important à savoir pour le jury : mon IA de Puissance 4 n'est **PAS** un réseau de neurones. C'est un **algorithme de recherche : minimax + élagage alpha-bêta**. Si on me demande « c'est de l'IA / du deep learning ? », je réponds : *« C'est de l'IA au sens **algorithme de décision** (minimax), pas de l'apprentissage automatique : il n'y a ni données d'entraînement ni réseau de neurones. »*

a) C'est quoi un réseau de neurones ? (culture générale)

Un **réseau de neurones** est un modèle d'**apprentissage automatique** (machine learning) : des « neurones » artificiels organisés en **couches** (entrée → couches cachées → sortie). Chaque connexion a un **poids** ; on

entraîne le réseau sur des **milliers d'exemples** en ajustant ces poids (par **rétropropagation**) pour minimiser l'erreur. Une fois entraîné, il **prédit** une sortie pour une nouvelle entrée.

→ Pour Puissance 4, un réseau de neurones devrait être **entraîné** (ex. par des millions de parties, comme AlphaZero). **C'est lourd, nécessite des données et de la puissance** : surdimensionné et inadapté à un Raspberry Pi hors-ligne.

b) Ce que j'utilise vraiment : minimax + alpha-bêta

Pas d'entraînement, pas de données : l'IA **raisonne en direct** en simulant les coups.

1. **Minimax** : on construit l'**arbre des coups possibles**. À mon tour je **maximise** mon score ; je suppose que l'adversaire **minimise** le mien. On descend jusqu'à une **profondeur** donnée, puis on **évalue** la position.
▶ `function minimax(grid, depth, alpha, beta, maximizing): number { ... }`
2. **Évaluation (heuristique)** : pas besoin d'aller jusqu'à la fin de partie ; on **note** la position avec `scoreWindow` (chaque alignement de 4 cases).
▶ `if (me === 4) return WIN_SCORE; · if (opp === 3) return -170; *(la défense prime)*`
3. **Élagage alpha-bêta** : on **coupe** les branches qui ne peuvent pas changer la décision → on explore **beaucoup plus profond** dans le même temps.
▶ `if (alpha >= beta) break; // coupure`
4. **Ordre des coups** : on examine **le centre d'abord** (colonnes les plus fortes) pour que l'élagage coupe davantage.
▶ `const COL_WEIGHT = [1, 2, 4, 7, 4, 2];`
5. **Niveaux de difficulté** = **profondeur** + un peu de hasard (« bruit ») pour les niveaux faibles.
▶ `novice: { depth: 1, noise: 0.70 } ... legendaire: { depth: 12, noise: 0.00 }`
6. **Anti-piège** : l'IA ne joue jamais un coup qui **offre la victoire** immédiate à l'adversaire (la défense `-170` l'emporte sur l'attaque `+130`).

c) Réseau de neurones vs minimax (tableau de réponse)

	Réseau de neurones	Minimax (mon choix)
Principe	apprend sur des données	calcule l'arbre des coups en direct
Entraînement	obligatoire (long, gourmand)	aucun
Données	des milliers de parties	zéro
Sur Raspberry Pi hors-ligne	lourd / inadapté	léger, instantané
Explicable	« boîte noire »	100 % explicable (je lis le code)

Phrase pour le jury : « J'ai choisi **minimax + alpha-bêta** plutôt qu'un **réseau de neurones** : pas besoin d'entraînement ni de données, c'est **léger, instantané et totalement explicable** sur un Raspberry Pi hors-ligne. L'IA simule les coups, suppose un adversaire optimal, et choisit le meilleur avec une **défense pondérée plus fort que l'attaque**. »

*Astuce d'oral : pour chaque terme, donne la **définition simple**, puis **montre la ligne de code** : c'est ça qui prouve que tu maîtrises ton projet.*

</content>